SIAM J. COMPUT.
Vol. 10, No. 3, August 1981

© 1981 Society for Industrial and Applied Mathematics
0097-5397/81/1003-0013 $01.00/0

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

PERFORMANCE BOUNDS FOR ORTHOGONAL ORIENTED TWO-DIMENSIONAL PACKING ALGORITHMS*

IGAL GOLAN†

Abstract. Two orthogonal oriented two-dimensional, packing algorithms are presented and their behavior is analyzed under several assumptions. For one of the algorithms we show that the ratio of the height used by the algorithm to the optimal height is asymptotically bounded by $\frac{4}{3}$. This bound is an improvement over similar bounds for previously proposed algorithms.

Key words. Two-dimensional packing

1. Introduction. Consider an "open-ended" rectangle R of width 1 and a finite collection of rectangular pieces organized into a list $L=\{P_1, P_2, \cdots, P_n\}$. Each piece is defined by an ordered pair $P_i=(w(P_i), h(P_i))$, $1 \leqq i \leqq n$, corresponding to the width $w(P_i)$ and the height $h(P_i)$ of the piece. $w(P_i) \leqq 1$ for all $1 \leqq i \leqq n$.

The problem is to pack the pieces into R so that no two pieces overlap and so that the height to which R is filled (the maximum height attained by any piece) is as small as possible. We shall assume that the rectangular pieces are oriented such that the sides of the piece which correspond to its width have to be parallel to the bottom edge (B) of R. The problem is a generalization of the one-dimensional bin packing problem studied in [4] and has been considered in various forms in [1], [2] and [3].

For a list L of rectangular pieces all having width less or equal to 1, let OPT(L) denote the minimum possible height necessary to pack all the pieces into R. Let $A(L)$ denote the height actually used by a packing algorithm A when applied to L. We consider two types of bounds: absolute performance bounds of the form $A(L) \leqq \alpha$ OPT(L) and asymptotic performance bounds of the form $A(L) \leqq \alpha$ OPT$(L)+\beta$, where α and β are constants such that $\alpha \geqq 1$, $\beta \geqq 0$.

In § 2 we present the split-algorithm and give absolute and asymptotic performance bounds for the general case, and for the special case where all pieces are squares. In § 3 we present the mixed-algorithm and give an asymptotic bound for its behavior.

We conjecture, on the basis of the analysis of these algorithms, and other algorithms as well, (such as those described in [1], [2]) that the better performance of the mixed-algorithm as compared to the split-algorithm is not accidental. More generally it appears that algorithms treating all the components of L uniformly are outperformed by those that handle "wide" and "narrow" pieces separately.

2. Performance bounds for split packing. We can *split* an "open-ended" rectangle R into two "open-ended" rectangles R_1, R_2 and a "closed" rectangle M as in Fig. 1. If B (the bottom edge of R) and Q coincide, M will disappear.

A *level* L will be any line parallel to the bottom edge B of R; its value is its distance from B. Let $L=\{P_1, P_2, \cdots, P_n\}$ be such that $w(P_i) \geqq w(P_{i+1})$ for $i<n$ (i.e., L is arranged in decreasing-width order). The *split algorithm* (SP-algorithm) will be as follows: the split algorithm packs the pieces in order of decreasing width. After packing some pieces, those left to be packed are narrower and we can split the open-ended

* Received by the editors August 16, 1979, and in final form February 21, 1980.
† Armament Development Authority, Center for Military Analyses, P.O. Box 2250, Haifa, Israel.

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

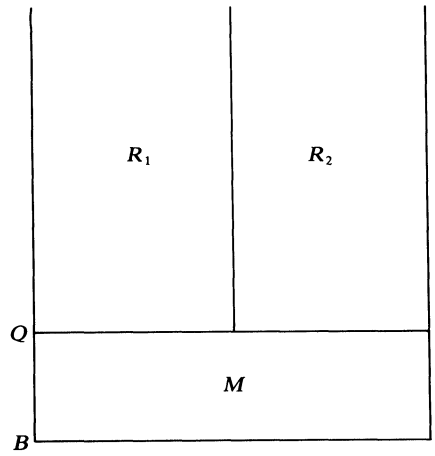

FIG. 1. *Splitting R into R_1, R_2 and M.*

rectangle R into two open-ended rectangles each being sufficiently wide as to accommodate any of the pieces left to be packed. Packing the two rectangles simultaneously, we insert the next piece to be packed in the one with the minimum resulting overall height. As soon as possible we continue to split each of the rectangles. As the pieces get narrower we obtain more rectangles to place them in.

Begin SP-Algorithm

(1) Define R_1 to be R. Place P_1 on B touching the left edge of R_1. Define level $S=0$, $k=1$, $i=1$.

While $(k<n)$ **Repeat**

Begin We have already defined $R_1, R_2, \cdots, R_i$. For each R_j with width W_j let P_j be the last piece packed into it. Let b_j be the distance of the bottom edge of P_j to B. Let $a_j = b_j + h(P_j)$.

(2) Let $J = \{R_j \mid W_j \geqq w(P_j) + w(P_{k+1})\}$

Comment J is the set of "open-ended" rectangles which can accomodate the next piece to be placed (P_{k+1}) side by side with the last piece placed in it (P_j).

If $(J=\Phi)$ **go to** (8)

(3) Find $R_{j'} \in J$ such that $b_{j'} \leqq b_j$ for all $R_j \in J$ (Resolve ties in any way you like).

(4) Insert P_{k+1} in $R_{j'}$ with its bottom edge at height $b_{j'}$ and its left edge touching $P_{j'}$ on its right.

(5) Split $R_{j'}$ into R_{i+1}, R_{i+2} and $M_{j'}$ with a line parallel to the bottom edge of $R_{j'}$ at height $b_{j'}$ and a vertical line touching $P_{j'}$ and P_{k+1} (see Fig. 2).

(6) Delete $R_{j'}$ from the list of "open-ended" rectangles. Put R_{i+1} and R_{i+2} in the list. Relabel the List. $i=i+1$, $k=k+1$.

(7) **Go to** (10).

(8) Let j' be such that $a_{j'} \leqq a_j$ for all $1 \leqq j \leqq i$ (resolve ties in any way you like). Place P_{k+1} in $R_{j'}$ with its bottom edge on the top edge of $P_{j'}$ and its left edge touching the left edge of $R_{j'}$.

(9) Define $S = a_{j'}$, $k=k+1$.

(10) **End** while.

(11) **Stop**

End SP-Algorithm.

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

LEMMA 1. *For any "closed" rectangle M let $P_{k_1}, P_{k_2}, \cdots, P_{k_f}$ be the pieces of L inside M. Let the width of M be $\bar{W}$. Then for all $1 \leqq i \leqq f$ $w(P_{k_i}) > \bar{W}/2$.*

Proof. The pieces inside M are placed on top of one another. If P_{k_i} is the piece placed on top of $P_{k_{i-1}}$, $1 < i \leqq f$ then $w(P_{k_i}) \leqq w(P_{k_{i-1}})$. If $f = 1$ we have $w(P_{k_1}) = \bar{W}$. If $f \geqq 2$ let us assume that $w(P_{k_f}) \leqq \bar{W}/2$. In this case a piece would be placed side by side with P_{k_f} using step (4) of SP and the split in step (5) would separate the pieces $P_{k_{f-1}}$ and P_{k_f}. This is a contradiction and therefore, $w(P_{k_f}) > \bar{W}/2$. □

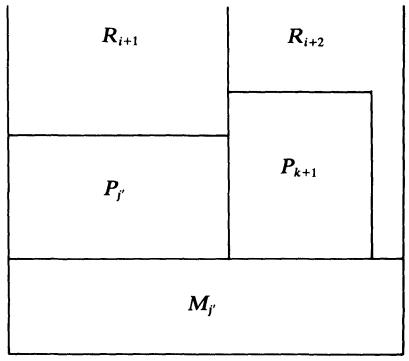

FIG. 2. *Splitting $R_{j'}$.*

LEMMA 2. *Let A be the region between B and S. Then A is at least half occupied with pieces (and parts of pieces) from L.*

Proof. Initially the level S is at the bottom edge of R and obviously the lemma is true. As long as we do not perform instruction (9) in the SP-algorithm the statement will remain true. Let us assume that we have just executed instruction (9) and S is driven to a new height. Between B and S we have "closed" rectangles (and parts of "closed" rectangles) and by Lemma 1 their area is at least half occupied. In addition we have in A parts of "open-ended" rectangles $R_{i_1}, R_{i_2}, \cdots, R_{i_\alpha}$. For all $1 \leqq j \leqq \alpha$, $S \leqq a_{i_j}$ (by step (8) of the SP-Algorithm) and $b_{i_j} > W_{i_j}/2$, for otherwise, at step (2), $J \neq \Phi$ and we would not have arrived at step (9). It follows that these parts are also half occupied. □

LEMMA 3. *If $h = \max_{1 \leqq i \leqq n'} (h(P_i))$ then for all R_j as defined by the* SP-*algorithm, $a_j - S \leqq h$.*

Proof. Assume that there is an i such that $a_i - S > h$. It follows that $b_i - S > 0$ and the P_i was inserted by using step (8). Then we would have used step (9) to move S to the bottom of P_i and $a_i - S = h(P_i)$. Thus, $a_i - S \leqq h$, a contradiction and the lemma is proved. □

THEOREM 1. *For any list $L = \{P_1, \cdots, P_n\}$, $\mathrm{SP}(L) \leqq 3\,\mathrm{OPT}(L)$.*

Proof.

$$\mathrm{SP}(L) \leqq S + h, \qquad h = \max_{1 \leqq i \leqq n} (h(P_i)) \quad \text{by Lemma 3,}$$

$$S \leqq 2 \sum_{i=1}^{n} w(P_i) * h(P_i) \quad \text{by Lemma 2,}$$

$$\mathrm{OPT}(L) \geqq \max\left(h, \sum_{i=1}^{n} w(P_i) * h(P_i)\right).$$

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Consider the two cases.

(1)
$$h \geqq \sum_{i=1}^{n} w(P_i) * h(P_i),$$
$$\mathrm{OPT}(L) \geqq h, \qquad \frac{\mathrm{SP}(L)}{\mathrm{OPT}(L)} \leqq \frac{S+h}{h} \leqq 1 + \frac{2\sum_{i=1}^{n} w(P_i) * h(P_i)}{h},$$
$$\frac{\mathrm{SP}(L)}{\mathrm{OPT}(L)} \leqq 3.$$

Thus, $\mathrm{SP}(L) \leqq 3\ \mathrm{OPT}(L)$.

(2)
$$h < \sum_{i=1}^{n} w(P_i) * h(P_i),$$
$$\mathrm{OPT}(L) \geqq \sum_{i=1}^{n} w(P_i) * h(P_i), \qquad \frac{\mathrm{SP}(L)}{\mathrm{OPT}(L)} \leqq \frac{S+h}{\sum_{i=1}^{n} w(P_i) * h(P_i)},$$
$$\frac{\mathrm{SP}(L)}{\mathrm{OPT}(L)} \leqq \frac{2\sum_{i=1}^{n} w(P_i) * h(P_i)}{\sum_{i=1}^{n} w(P_i) * h(P_i)} + \frac{h}{\sum_{i=1}^{n} w(P_i) * h(P_i)} \leqq 3.$$

Therefore, $\mathrm{SP}(L) \leqq 3\ \mathrm{OPT}(L)$. □

COROLLARY 1. *For any list* $L = \{P_1, P_2, \cdots, P_n\}$ *such that* $w(P_i), h(P_i) \leqq 1$ *for all* $1 \leqq i \leqq n$, $\mathrm{SP}(L) \leqq 2\ \mathrm{OPT}(L) + 1$.

Proof.
$$\mathrm{SP}(L) \leqq S + h, \qquad h = \max_{1 \leqq i \leqq n} (h(P_i)) \leqq 1,$$
$$S \leqq 2 \sum_{i=1}^{n} w(P_i) * h(P_i) \leqq 2\ \mathrm{OPT}(L).$$

Therefore, $\mathrm{SP}(L) \leqq 2\ \mathrm{OPT}(L) + 1$. □

Example 1. For any even k let $L = \{P_i \mid i = 1, \cdots, k^2/2 + k^3/2 + k + 1\}$, where
$$1 \leqq j \leqq k^2/2, \qquad P_j = (2/k^2 - \varepsilon, 2/k^2 - \varepsilon),$$
$$n_1 = \frac{k^2}{2} + 1 \leqq j \leqq \frac{k^2}{2} + \frac{k^3}{2} + k, \qquad P_j = \left(\frac{1}{k^2}, \frac{1}{k^2}\right),$$
$$n = \frac{k^2}{2} + \frac{k^3}{2} + k + 1, \qquad P_n = \left(\frac{1}{k^2} - \varepsilon, \frac{1}{2k}\right), \quad \varepsilon \ll \frac{2}{k^4}.$$

Then an SP-packing of L will be as in Fig. 3, and the optimal packing will be as in Fig. 4.
$$\mathrm{SP}(L) = \frac{2}{k^2} + \frac{3}{2k} - \varepsilon, \qquad \mathrm{OPT}(L) = \frac{3}{k^2} + \frac{1}{2k} - \varepsilon, \qquad \frac{\mathrm{SP}(L)}{\mathrm{OPT}(L)} = 3 - \frac{7 - 2\varepsilon k^2}{3 + \frac{k}{2} - \varepsilon k^2}.$$

COROLLARY 2. *For any* $\delta > 0$ *there exists a list* L *of rectangles arranged by decreasing width, such that* $\mathrm{SP}(L) > (3 - \delta)\ \mathrm{OPT}(L)$.

Proof. Use Example 1. □

Example 2. For any even k let $L=\{P_i \mid i=1,\cdots,(k^5/2)+k^3+(k^2/2)\}$ where

$$1 \leqq j \leqq \frac{k^2}{2}, \qquad P_j=\left(\frac{2}{k^2}-\varepsilon, \frac{2}{k^2}-\varepsilon\right),$$

$$n_1=\frac{k^2}{2}+1 \leqq j \leqq \frac{k^5}{2}+k^3+\frac{k^2}{2}, \qquad P_j=\left(\frac{1}{k^2}, \frac{1}{k^2}\right), \quad \text{and} \quad \varepsilon \ll \frac{2}{k^4}.$$

The SP-packing will be similar to that in Fig. 3, and the optimal packing will be similar to that in Fig. 4.

$$\mathrm{SP}(L)=\left(\frac{2}{k^2}-\varepsilon\right)+k^3 * \frac{1}{k^2}, \qquad \mathrm{SP}(L)=k+\frac{2}{k^2}-\varepsilon,$$

$$\mathrm{OPT}(L)=\left(\frac{k^3}{2}+k\right) * \frac{1}{k^2}+\left(\frac{2}{k^2}-\varepsilon\right), \qquad \mathrm{OPT}(L)=\frac{k}{2}+\frac{1}{k}+\frac{2}{k^2}-\varepsilon.$$

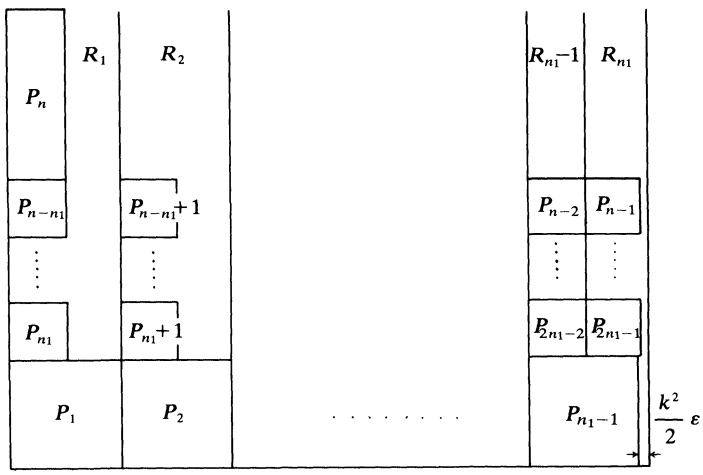

FIG. 3. $\mathrm{SP}(L)=2/k^2+3/2k-\varepsilon$.

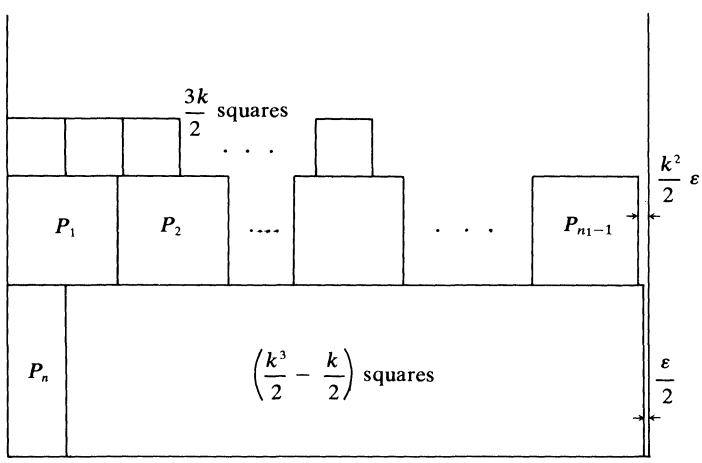

FIG. 4. $\mathrm{OPT}(L)=3/k^2+1/2k-\varepsilon$.

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

COROLLARY 3. *For any $\delta > 0$ there exists a list L of rectangles arranged by decreasing width, such that* $\mathrm{SP}(L) > (2-\delta)\mathrm{OPT}(L) + C$ *for any C.*

Proof. Use Example 2.

$$\frac{\mathrm{SP}(L)-C}{\mathrm{OPT}(L)} = \frac{k+\frac{2}{k^2}-C-\varepsilon}{\frac{k}{2}+\frac{1}{k}+\frac{2}{k^2}-\varepsilon} = 2-\frac{4+2Ck^2+4k-2k^2\varepsilon}{k^3+2k+4-2k^2\varepsilon}.$$

If k is large enough, $(\mathrm{SP}(L)-C)/(\mathrm{OPT}(L)) > 2-\delta$. □

When all the pieces are squares an improvement of the SP-algorithm can be obtained.

The new algorithm, called the SPS-algorithm, does not insert the squares into R in the order they appear in L. To get the SPS-algorithm replace step (8) in the SP-algorithm by the following steps:

(8′) **Begin** *Let* j' be such that $a_{j'} \leqq a_j$ for all $1 \leqq j \leqq i$ (resolve ties in any way you like). Let $d = 0$.

(81) $e = W_{j'} - w(P_{j'}) - d$. If there is no m, $m > k+1$, such that $w(P_m) \leqq e$ **go to** (82). Let l, $l > k+1$, be the smallest integer for which $w(P_l) \leqq e$. Insert P_l in $R_{j'}$ with its bottom edge at distance $b_{j'}$ from B, and as far to the left as possible. Let $d = d + w(P_l)$. Take P_l out of the list L, and relabel the remaining pieces of L starting at $k+1$. Let $n = n-1$, **go to** (81).

(82) Insert P_{k+1} in $R_{j'}$ with its bottom edge on the top edge of $P_{j'}$ and its left edge touching the left edge of $R_{j'}$.

End

THEOREM 2. *For any list $L = \{P_1, P_2, \cdots, P_n\}$ of squares arranged by decreasing-width* $\mathrm{SPS}(L) \leqq 2\ \mathrm{OPT}(L)$.

Proof. Let P_t be the largest square with top at height $\mathrm{SPS}(L)$. Define level $S' = w(P_t)$ and level $S_1 = \mathrm{SPS}(L) - w(P_t)$. By the definition of S, $S \geqq S_1$. Let A be the area between levels S' and S_1. Lemma 2 holds also for the SPS-algorithm and hence, A is at least half occupied.

Consider the area A_1 between B and S'. There are pieces of L on B arranged in decreasing width order from left to right. Let P_q be the square farthest to the right which is still greater than P_t. The distance z between the right edge of P_q and the right edge of R, is less than $w(P_t)$, otherwise, P_t would have been placed on B using step (81). The occupied area of A_1 is at least $w(P_t) * (1-z)$. Consider the area A_2 between S_1 and $\mathrm{SPS}(L)$. The occupied area of A_2 is at least $w(P_t) * w(P_t)$. The occupied area of both A_1 and A_2 is at least half the area, because $w(P_t) * (1-z) + w(P_t) * w(P_t) \geqq w(P_t)$. Thus, $\mathrm{SPS}(L) \leqq 2\ \mathrm{OPT}(L)$. □

COROLLARY 4. *For any $\delta > 0$, there exists a list L of squares arranged by decreasing width, such that* $\mathrm{SPS}(L) > (2-\delta)\mathrm{OPT}(L)$.

Proof. Use Example 1 without the last piece. □

3. Performance bound for mixed packing. Most of the packing algorithms (see [1], [2] and [3]) packed all the pieces in the same way, an exception is [2, split-fit algorithm]. The mixed algorithm divided the pieces into several sets and packed the pieces in the sets in different ways. By not packing the pieces in decreasing-width order, it is possible to fill the gaps created by packing wide pieces, with smaller pieces and thus save space.

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Given a list $L=\{P_1, P_2, \cdots, P_n\}$ of rectangular pieces, such that $w(P_i), h(P_i) \leqq 1$, for all $1 \leqq i \leqq n$, divide L into 5 sets: $A=\{P_i \mid w(P_i)>\frac{1}{2}\}$, $B=\{P_i \mid \frac{1}{3}<w(P_i) \leqq \frac{1}{2}\}$, $C=\{P_i \mid \frac{1}{4}<w(P_i) \leqq \frac{1}{3}\}$, $D_1=\{P_i \mid \frac{5}{24}<w(P_i) \leqq \frac{1}{4}\}$, $D_2=\{P_i \mid w(P_i) \leqq \frac{5}{24}\}$ and let $D=D_1 \cup D_2$. A piece from the A set will be called an A-piece and similarly for B, C, D_1, D_2 and D.

The mixed-algorithm (M-algorithm) is as follows:

Begin M-algorithm.

Step (1)

(1.1) Arrange all A-pieces in decreasing width order; $A=\{a_1, a_2, \cdots, a_{|A|}\}$. Set $l_1=0$, $i=1$.

(1.2) Place a_i in R on level l_i left justified (as far to the left edge of R as possible).

(1.3) $l_{i+1}=l_i+h(a_i)$, $i=i+1$.

(1.4) **If** $i \leqq |A|$ **then go to** (1.2).

Step (2)

(2.1) Define $L_2=l_{|A|+1}$. Arrange all B-pieces in decreasing width order; $B=\{b_1, b_2, \cdots, b_{|B|}\}$. Set level $f=0$.

(2.2) Let

$$k=\min_{1 \leqq j \leqq |B|}\{|B|+1, j \mid b_j \in B,\ b_j \text{ is not marked},\ b_j \text{ will fit on level } f, f+h(b_j) \leqq L_2\}.$$

If $(k=|B|+1)$ **then go to** (2.4).

(2.3) Place b_k in R on level f right justified. Mark b_k, $f=f+h(b_k)$, **go to** (2.2).

(2.4) Find $i=\min_{1 \leqq j \leqq |A|+1}(j \mid l_j>f)$.
If $(i \leqq |A|)$ **then** $f=l_i$, **go to** (2.2).

(2.5) Move up the last B-piece placed in R, until its top will be at level L_2. Move, successively, all other B-pieces, until the top of each one will touch the bottom of the B-piece above it. Let L_1 be the height of the bottom of the lowest B-piece in R.

Step (3)

(3.1) Find $i=\max_{1 \leqq j \leqq |A|+1}(0, j \mid w(a_j)>\frac{3}{4})$. Let $H_1=l_{i+1}$, **if** $(H_1 \geqq L_1)$ **then go to** (6). Find $i=\max_{1 \leqq j \leqq |A|+1}(0, j \mid w(a_j)>\frac{2}{3})$. Let $H_2=\min(L_1, l_{i+1})$. Find $i=\max_{1 \leqq j \leqq |A|+1}(0, j \mid w(a_j)>\frac{7}{12})$. Let $H_3=\min(L_1, l_{i+1})$.

Step (4)

(4.1) **If** $(H_2=H_1)$ **then go to** (5). Construct a rectangle R_1 inside R, between height H_1 and height H_2, having width $\frac{1}{4}$, right justified (see Fig. 5). Use FFDH-algorithm [2], to pack as many of the remaining pieces as possible into R_1. If a piece does not fit completely into R_1, then don't pack it.

(4.2) **If** $(H_3=H_2)$ **then go to** (5). Construct a rectangle R_2 inside R, between height H_2 and height H_3, having width $\frac{1}{3}$ right justified (see Fig. 5). Use FFDH-algorithm [2], to pack as many of the remaining pieces as possible into R_2. If a piece does not fit completely into R_2, then don't pack it.

Step (5)

(5.1) Arrange all C-pieces not yet packed in decreasing-width order; $C=\{c_1, c_2, \cdots, c_{|C|}\}$. $H_4=H_3$, $i=1$.

(5.2) **If** $(i>|C|)$ **then go to** (5.5).

(5.3) **If** $(H_4+h(c_i)>L_1)$ **then** $i=i+1$, **go to** (5.2).

(5.4) Put c_i in R on level H_4 right justified, $H_4=H_4+h(c_i)$, $i=i+1$, **go to** (5.2).

(5.5) **If** $(H_4+1 \geqq L_1)$ **then go to** (6).

(5.6) Arrange all D_1-pieces not yet packed in decreasing-width order; $D_1 = \{d_1, d_2, \cdots, d_{|D_1|}\}$. Use [1, bottom up right justified algorithm] to pack as many of the D_1-pieces as possible, starting at height H_4, but do not pack above L_1. Let the height of the top of the highest D_1-piece be H_5.

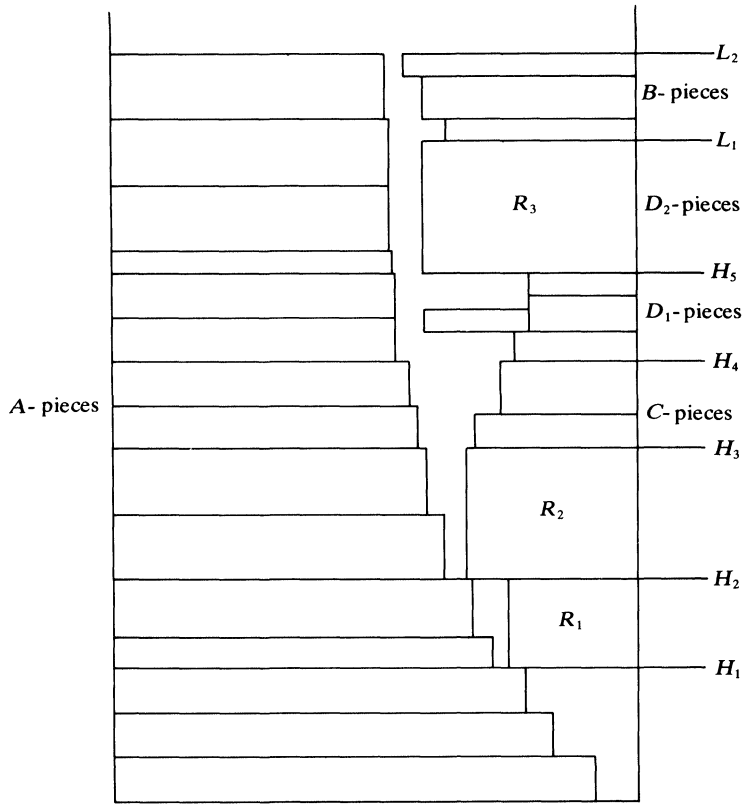

FIG. 5. *M-packing steps* (1)–(5).

(5.7) **If** $(H_5 + 1 \geqq L_1)$ **then go to** (6).

(5.8) Construct a rectangle R_3 inside R, between height H_5 and height L_1, having width $\frac{5}{12}$ right justified (see Fig. 5). Use FFDH-algorithm [2], to pack as many of the remaining D_2-pieces as possible into R_3. If a piece does not fit completely into R_3, then don't pack it.

Step (6)

(6.1) Use FFDH-algorithm [2] to pack the remaining B-pieces left justified. The pieces will be packed starting at height L_2 and ending at height L_4.

(6.2) Rearrange the levels of this part of the packing in such a way that all levels having total width more than $\frac{3}{4}$ will be below all other levels. Let L_3 be the highest level having total width more than $\frac{3}{4}$.

Step (7)

(7.1) **If** $(L_3 = L_4)$ **then go to** (8). Construct a rectangle R_4 inside R, between height L_3 and height L_4, having width $\frac{1}{4}$ right justified. Use FFDH-algorithm [2], to pack as many of the remaining pieces as possible into R_4. If a piece does not fit completely into R_4, then don't pack it.

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Step (8)

(8.1) Use FFDH-algorithm [2], to pack all remaining pieces left justified, starting at height L_4.

End M-algorithm.

THEOREM 3. *For any list* $L=\{P_1, \cdots, P_n\}$, *such that* $w(P_i)$, $h(P_i) \leqq 1$ *for all* $1 \leqq i \leqq n$, $M(L) \leqq \frac{4}{3}\,\mathrm{OPT}(L)+7\frac{1}{18}$.

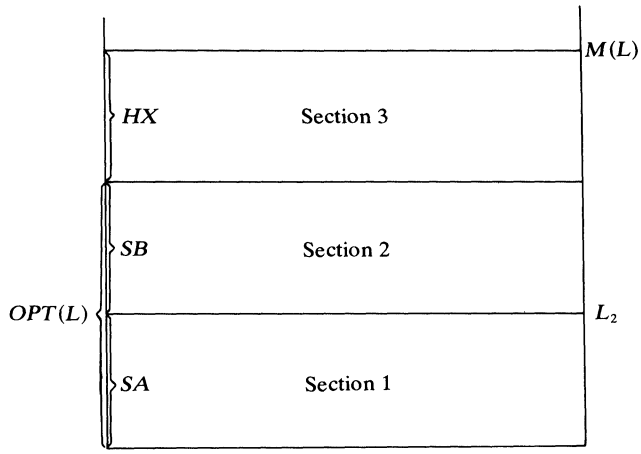

FIG. 6. *M-packing divided into sections.*

Proof.

Case 1. *No* D-*piece left to be packed after step* (7). Let OPT(L) be the height of the optimal packing. Divide the optimal packing into blocks, by passing levels at the top and bottom of each A-piece. Each adjacent pair of such levels defines a block. Let a block containing an A-piece be called an A-block, and any other block a B-block. Let SA be the sum of the heights of all A-blocks and SB the sum of the heights of all B-blocks. Then

$$\mathrm{OPT}(L) = SA + SB.$$

Let $H(Y)$ be the sum of the heights of all Y-pieces, where Y is A, B or C. Let $HH = H(A)+H(B)+H(C)$. It is clear that $SA = H(A) = L_2$.

Divide the M-packing into 3 sections as follows: section 1 between the bottom of R and L_2, section 2 between L_2 and OPT(L) and section 3 between OPT(L) and $M(L)$ (Fig. 6). Let $X(B)$ be the sum of the heights of all B-pieces or parts of B-pieces, included in section 3. Let $X(C)$ be the sum of the heights of all C-pieces or parts of C-pieces, included in section 3. Let HX be the height of section 3, $M(L) = \mathrm{OPT}(L)+HX$. Clearly the height of section 2 is SB.

In any packing, a level line can pass through at most 3 pieces from $B \cup C$. If a level line passes through an A-piece, it can pass, at most, through one more piece from $B \cup C$. Thus $HH \leqq 2SA+3SB$.

In the second section of the M-packing, FFDH-packing was used in steps (6) and (8). In each block of the FFDH-packing, in steps (6) and (8), we have at least two pieces from $B \cup C$, except maybe, the last block in each step. In a FFDH-packing of height L, if each block, except possibly the last one, contains at least K pieces, then the sum S of the heights of all pieces, satisfies $S \geqq KL-(L-1)H$, H being the height of the highest piece.

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Hence, $X(B)+X(C) \leqq H(B)+H(C)-(2SB-2)$. But $H(B)+H(C)=HH-H(A)$, and $HH \leqq 2SA+3SB$. Therefore $X(B)+X(C) \leqq 2SA+3SB-SA-2SB+2$. Hence $X(B)+X(C) \leqq SA+SB+2$. But $\mathrm{OPT}(L)=SA+SB$. Thus $X(B)+X(C) \leqq \mathrm{OPT}(L)+2$.

Compare the packing of the B-pieces in the M-packing with that in the optimal packing. In step (2) of the M-algorithm, B-pieces are placed side by side with A-pieces. The B-pieces are ordered in decreasing-width order (2.1), and the k in step (2.2) is minimal, thus we always place the current widest B-piece. The cumulative height of the B-pieces put in step (2) can differ from the maximum possible only because of the additional condition $f+h(b_j) \leqq L_2$ in step (2.2). This implies that there might be an additional B-piece that will fit near an A-piece, but will violate the above restriction. We conclude that the cumulative height of the B-pieces placed in the first section is not less than the cumulative height of the B-pieces, and parts of B-pieces, in the A-blocks minus 1. If $\mathrm{OPT}(L) \geqq L_4$ we have $X(B)=0$. Assume $\mathrm{OPT}(L)<L_4$; then the cumulative height of the B-pieces, and parts of B-pieces, placed in step (6) up to height $\mathrm{OPT}(L)$, is at least $2SB-1$. This is because the FFDH-algorithm will put 2-pieces in each level, except maybe the last one. The cumulative height of the B-pieces, and parts of B-pieces, packed by the optimal algorithm in the B-blocks, is not more than $2SB$. Therefore, $X(B) \leqq 2$.

Divide the third section of the M packing into blocks, by passing levels at the top and bottom of each B-piece, or part of a B-piece. Each block will be between two near levels. Call a block containing a B-piece, or part of a B-piece, a BM-block and any other block a CM-block. Let SBM be the sum of the heights of all BM-blocks, and SCM the sum of the heights of all CM-blocks. Then

$$HX = SBM + SCM, \qquad SBM \leqq X(B) \leqq 2.$$

Using step (8) we pack the C-pieces, placing them three on a level except maybe the last one. Thus

$$SCM \leqq \frac{X(C)}{3}+2, \qquad HX \leqq \frac{X(C)}{3}+4.$$

But $X(B)+X(C) \leqq \mathrm{OPT}(L)+2$. Therefore $HX \leqq (\mathrm{OPT}(L))/3+4\frac{2}{3}$,

$$M(L)=\mathrm{OPT}(L)+HX \leqq \tfrac{4}{3}\mathrm{OPT}(L)+4\tfrac{2}{3}.$$

Case 2. *There are D-pieces left to be packed after step* (7). We will use an area size argument in this case. In each of the 10 parts of the proof we will show that the area S, between two line levels, covered by pieces, satisfies $S \geqq \alpha H+\beta$ where $\alpha \geqq \frac{3}{4}$, H the distance between the levels, and β a constant.

(i) From the bottom of R to level H_1, the area S_1 covered by the A-pieces satisfies $S_1 \geqq \frac{3}{4}H_1$ (definition of H_1 in step (3) of the M-algorithm).

(ii) From level H_1 to level H_2, the area S_2' covered by the A-pieces satisfies $S_2' \geqq \frac{2}{3}(H_2-H_1)$ (definition of H_2 in step (3) of the M-algorithm). The area S_2'' covered by pieces inside R_1 satisfies $S_2'' \geqq \frac{1}{4}((H_2-H_1)/2-1)$ ([2, Thm. 1] and

$$S_2=S_2'+S_2'' \geqq (\tfrac{2}{3}+\tfrac{1}{8})(H_2-H_1)-\tfrac{1}{4} \geqq \tfrac{3}{4}(H_2-H_1)-\tfrac{1}{4}.$$

(iii) From level H_2 to level H_3, the area S_3' covered by the A-pieces satisfies $S_3' \geqq \frac{7}{12}(H_3-H_2)$ (definition of H_3 in step (3) of the M-algorithm). The area S_3'' covered by pieces inside R_2 satisfies $S_3'' \geqq \frac{1}{3}((H_3-H_2)/2-1)$ [2, Thm. 1].

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Thus,

$$S_3 = S_3' + S_3'' \geqq (\tfrac{7}{12}+\tfrac{1}{6})(H_3 - H_2) - \tfrac{1}{3} \geqq \tfrac{3}{4}(H_3 - H_2) - \tfrac{1}{3}.$$

(iv) From level H_3 to level H_4, the area S_4' covered by the A-pieces satisfies $S_4' \geqq \frac{1}{2}(H_4 - H_3)$. The area S_4'' covered by the C-pieces satisfies $S_4'' \geqq \frac{1}{4}(H_4 - H_3)$. Thus,

$$S_4 = S_4' + S_4'' \geqq (\tfrac{1}{2}+\tfrac{1}{4})(H_4 - H_3) \geqq \tfrac{3}{4}(H_4 - H_3).$$

(v) Look at any level line between levels H_4 and $H_5 - 1$. It passes through an A-piece on the left, and through a D_1-piece on the right. In between there is a gap filled with at most one additional D_1-piece. If the width of the gap is at least $\frac{1}{4}$, then because the D_1-pieces have width less than $\frac{1}{4}$, the bottom up right justified algorithm used in step (5.6) will certainly place another D_1-piece into the gap. Let S_5 be the area covered by pieces from level H_4 to level H_5, then

$$S_5 \geqq \tfrac{3}{4}((H_5 - 1) - H_4) + \tfrac{1}{2} \geqq \tfrac{3}{4}(H_5 - H_4) - \tfrac{1}{4}.$$

(vi) From level H_5 to level L_1, the area S_6' covered by the A-pieces, satisfies $S_6' \geqq \frac{1}{2}(L_1 - H_5)$. The area S_6'' covered by pieces inside R_3 satisfies $S_6'' \geqq \frac{5}{12}(\frac{2}{3}(L_1 - H_5) - 2\frac{1}{4} \cdot \frac{2}{3})$. (The width of R_3 is $\frac{5}{12}$. The D_2-pieces used to pack R_3 have a width of at most $\frac{5}{24}$. We use a result from the proof [2, Thm. 3] for the bound.) Thus,

$$S_6 = S_6' + S_6'' \geqq (\tfrac{1}{2}+\tfrac{10}{36})(L_1 - H_5) - \tfrac{5}{8} \geqq \tfrac{3}{4}(L_1 - H_5) - \tfrac{5}{8}.$$

(vii) From level L_1 to level L_2, the area S_7' covered by the A-pieces satisfies $S_7' \geqq \frac{1}{2}(L_2 - L_1)$. The area S_7'' covered by the B-pieces satisfies $S_7'' \geqq \frac{1}{3}(L_2 - L_1)$. Thus,

$$S_7 = S_7' + S_7'' \geqq (\tfrac{1}{2}+\tfrac{1}{3})(L_2 - L_1) \geqq \tfrac{3}{4}(L_2 - L_1).$$

(viii) From level L_2 to level L_3, the area S_8 covered by the B-pieces satisfies $S_8 \geqq \frac{3}{4}(L_3 - L_2) - 1$ (definition of L_3 in step (6) of the M-algorithm).

(ix) From level L_3 to level L_4, the area S_9' covered by the B-pieces satisfies $S_9' \geqq \frac{2}{3}(L_4 - L_3) - 1$. (There are 2 B-pieces in each level except possibly the last one.) The area S_9'' covered by pieces inside R_4 satisfies $S_9'' \geqq \frac{1}{4}((L_4 - L_3)/2 - 1)$ [2, Thm. 1]. Thus,

$$S_9 = S_9' + S_9'' \geqq (\tfrac{2}{3}+\tfrac{1}{8})(L_4 - L_3) - \tfrac{5}{4} \geqq \tfrac{3}{4}(L_4 - L_3) - \tfrac{5}{4}.$$

(x) All pieces placed from level L_4 to the end of the packing $M(L)$ have width less than $\frac{1}{3}$. Using again the result from the proof of [2, Thm. 3] we get: $S_{10} \geqq \frac{3}{4}(M(L) - L_4) - \frac{19}{9} \cdot \frac{3}{4}$. Thus,

$$S_{10} \geqq \tfrac{3}{4}(M(L) - L_4) - \tfrac{19}{12}.$$

Adding the results from (i) to (x) we get $\mathrm{OPT}\,(L) \geqq \sum_{i=1}^{10} S_i \geqq \frac{3}{4}M(L) - \frac{127}{24}$. Thus, $M(L) \leqq \frac{4}{3}\mathrm{OPT}(L) + 7\frac{1}{18}$. $\square$

REFERENCES

[1] B. Baker, E. G. Coffman and R. L. Rivest, *Orthogonal packings in two dimensions*, this Journal, 9 (1980), pp. 846–855.

[2] E. G. COFFMAN, M. R. GAREY, D. S. JOHNSON AND R. E. TARJAN, *Performance bounds for level-oriented two dimensional packing algorithms*, TR-CSL-7808 UCSB Santa Barbara, Ca. 93106; this Journal, 9 (1980), pp. 808–826.

[3] I. GOLAN, *Two Orthogonal Oriented Algorithms for Packing in Two Dimensions*, 1979/311/MHM Computer Center M.O.D., P.O. Box 2250, Haifa, Israel.

[4] D. S. JOHNSON, A. DEMERS, J. D. ULLMAN, M. R. GAREY AND R. L. GRAHAM, *Worst-case performance bounds for simple one-dimensional packing algorithms*, this Journal, 3 (1974), pp. 299–325.

Downloaded 12/06/25 to 14.139.128.40 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy